Product Specification

V1 — what we're shipping.

A complete, reviewer-approved specification of SynthData V1.
What it does, how it works, what ships, what doesn't.

V1 in one sentence.

A web platform where ML engineers generate synthetic text training datasets on demand — describe your domain, define your schema, bring your own API key, pick a sample count. We handle the rest.

Domain-agnostic

HR, fintech, medical, legal — any text domain

Schema-enforced

Every row validated against your JSON structure

BYOK

Your API key, your cost — no markup, no proxy

Here's what V1 can do.

Five core capabilities, measured and bounded.

100K

Max dataset size

samples per single job

6

Pipeline stages

Discover → Dedupe

5

AI providers supported

OpenAI, Anthropic, Groq, Bedrock, Ollama

5

NLP task types

intent, NER, Q&A, sentiment, custom

99%+

Schema validity target

with structured output mode

< 3min

Time to first dataset

for 1,000-sample jobs

What ships. What doesn't.

We cut aggressively. A focused V1 beats a bloated one.

IN V1

- Intent classification
- Named entity recognition
- Q&A pairs
- Sentiment classification
- Custom text schemas
- BYOK — 5 providers
- JSONL + JSON + CSV export
- Crash-safe resumable jobs
- MinHash dedup
- Chain-of-Thought judge

DEFERRED

- Tabular data → V2
- Multi-turn dialogue → V1.5
- Multilingual → V1.5
- Image/audio → never
- Team workspaces → V1.5
- OAuth/SSO → V1.1
- Dataset versioning → V1.5
- Downstream benchmarks → V2
- Augmentation (1M+) → V2
- Custom prompt templates → V1.5

Every deferred feature has a ship target. None are "maybe, someday."

THE PIPELINE

6 stages. One job each.

Stage 7 (augmentation) was dropped from V1 — reviewers flagged quality risk for NLP.

01

Discover

What varies in the domain?

02 NEW

Validate

Prune illogical combos

03

Compose

Build unique prompts

04

Generate

Parallel LLM calls + schema

05

Judge

Rules + CoT scoring

06

Dedupe

MinHash LSH (0.85)

Ask the AI what varies in your domain.

One LLM call to extract domain knowledge. Takes ~15 seconds.

INPUT

User provides:

- Domain: "HR Technology"
- Use case: "chatbot for hiring..."
- Schema: JSON structure
- Task type: intent classification

→

OUTPUT

LLM returns axes:

intent	→ 4 values
tone	→ 3 values
persona	→ 4 values
context	→ 3 values

Why this works: the LLM already knows the domain. We just ask it to tell us what varies.

STAGE 02 · VALIDATE · NEW

Kill the nonsense combinations.

A reviewer-requested addition. Prevents garbage combinations from reaching generation.

REJECTED

Illogical combinations:

```
intent: report_harassment
+ tone: cheerful
```

```
intent: emergency_alert
+ urgency: low_priority
```

```
intent: create_jd
+ context: retail_store
```

HOW IT WORKS

1. Generate all combinations
2. Batch to cheap LLM (50 at a time)
3. LLM marks each: yes or no
4. Drop anything marked "no"
5. Proceed only with valid combos

Typical pruning: 5-15% · Extra cost: ~2-3% · Worth it.

Turn combinations into prompts.

The allocation math lives here.

THE FORMULA

examples_per_combination = target_count ÷ valid_combinations

Sweet spot: 5 ≤ examples_per_combination ≤ 20

WORKED EXAMPLE — target 1,000 samples

Axes from Stage 1:	4 × 3 × 4 × 3 = 144 combinations
After Stage 2 pruning:	120 valid combinations
Allocation:	1,000 ÷ 120 ≈ 9 per combo
Plan total:	120 × 9 = 1,080 samples (5% oversample)

STAGE 04 · GENERATE

Parallel calls. Checkpointed to DB.

Every row persisted the moment it's validated. Crash = resume, not restart.

generation_plan

DB table

120 rows
status: pending

Worker pool

10 parallel
Celery tasks
concurrent LLM

Postgres

Write on
every success
complete flag

Promise to user: "If anything fails, you never lose what was generated. Resume always works."

Rules first. LLM second.

Two-layer quality check. Cheap layer first, expensive layer only if needed.

LAYER 1 — RULES

Deterministic. Free.

- ✓ Text length 5-500 chars
- ✓ Label in allowed set
- ✓ No placeholder text
- ✓ Enum fields valid
- ✓ Schema types match

LAYER 2 — LLM JUDGE

CoT prompted. Cheap model.

Judge writes reasoning, then scores:

Correctness	≥ 7
Realism	≥ 7
Distinctiveness	≥ 7

Judge failure → conservative pass

Free tier: layer 1 only. Pro/Enterprise: both layers.

MinHash — the middle-ground dedup.

Reviewer-recommended. Exact match is too weak. Semantic dedup is too aggressive.

EXACT MATCH

Too weak

"I want to create a JD."

→ *keep*

"I want to create a JD!"

→ *keep (just "!")*

"i WANT to create A JD"

→ *keep (casing)*

Misses trivial variants

MINHASH (V1)

Just right

"I want to create a JD."

→ *keep*

"I want to create a JD!"

→ *drop (same)*

"Need a new job listing"

→ *keep (paraphrase)*

Catches junk, keeps variety

SEMANTIC (V2)

Too aggressive

"I want to create a JD."

→ *keep*

"I want to create a JD!"

→ *drop*

"Need a new job listing"

→ *drop (too similar)*

Kills useful paraphrases

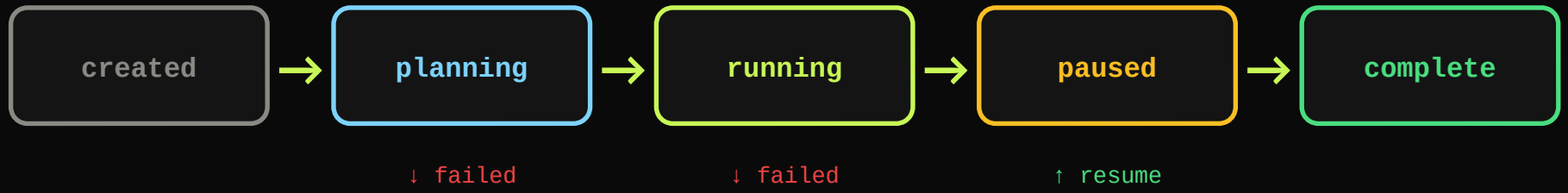
Things will break. Here's what happens.

Every failure mode has a defined response. No silent failures.

FAILURE MODE	V1 RESPONSE	SEVERITY
Network timeout	Auto-retry 3× with backoff	low
Rate limit (429)	Pause job, email user, auto-resume	med
Invalid API key	Fail job, clear message	high
Bad JSON output	Retry prompt 3×, skip if still bad	low
Safety refusal	Skip combo, log, continue	med
Judge hallucinates	Conservative pass (accept with flag)	low
Worker crashes	Task → pending, next worker picks up	low
DB outage	Pause job, alert ops, resume when back	high

State machine. Crash-safe by design.

Every state is persisted. Server can restart without losing any job.



REAL EXAMPLE

```
10:00  →  Job submitted (25,000 samples, OpenAI)
10:02  →  Planning complete, 250 combinations, generating...
12:30  →  Rate limit hit (429). Job paused. Email sent to user.
12:45  →  Auto-resume. Workers pick up pending rows from checkpoint.
13:55  →  Job complete. 24,847 rows. Email sent with download link.
```

Three tiers. Clear boundaries.

Pricing covers platform access. API costs stay with the user (BYOK).

FREE

$0

1,000 samples

3 min · ~$0 LLM cost

- Rule-based judge only
- 3 projects max
- 2 API keys
- Community support

PRO

$29 /month

25,000 samples

30 min · ~$2 LLM cost

- Full CoT judge
- Unlimited projects
- 10 API keys
- Email support

ENTERPRISE

Custom

100,000 samples

2-4 hrs · ~$20 LLM cost

- Priority workers
- Unlimited keys
- SLA + priority
- Onboarding help

QUALITY TARGETS

The numbers V1 promises.

If a job falls below these, it's flagged as degraded and the user is notified.

Schema validity

Rows passing schema check

≥ 98%

Judge pass rate

Rows meeting all 3 quality axes (Pro+)

≥ 85%

Unique row rate

Rows surviving MinHash dedup

≥ 95%

Label balance

Max deviation per label from target

≤ 20%

Time to first row

User sees output quickly

≤ 30s

Job resume success

Checkpointed jobs always resume

100%

Boring tech, battle-tested.

No clever choices. Everything here has been to production elsewhere.

BACKEND

- Python 3.11+
- FastAPI
- Celery + Redis
- PostgreSQL 16
- S3-compatible storage

FRONTEND

- Next.js 14
- React + Tailwind
- shadcn/ui
- Zustand + React Query
- Recharts

ML / DATA

- datasketch (MinHash)
- Pydantic v2
- Direct LLM SDKs
- No LangChain in prod
- No LiteLLM (security)

V1 is the beginning.

Every deferred feature has a home. We'll keep shipping.

V1.0

Now — 3 months

- 6-stage pipeline
- 100K samples max
- BYOK for 5 providers
- Crash-safe jobs

V1.1

+3 months

- OAuth (Google, GitHub)
- Multilingual (5 langs)
- Dynamic personas
- Team workspaces

V2

+9-12 months

- Tabular data engine
- Multi-turn dialogue
- Up to 1M samples safely
- Fine-tuning benchmarks

That's V1.
Ready to build.

Focused scope · Reviewer-approved architecture · Clear tiers

SynthData V1.0 · April 2026